

- Documentação do Sistema de Controle de Estoque
 - Sumário
 - 1. Visão Geral
 - Tecnologias
 - 2. Topologia da Arquitetura
 - 3. Diagrama de Camadas
 - 4. Modelo de Dados (MER)
 - Diagrama Entidade-Relacionamento
 - Dicionário de Dados
 - 5. Camada Model — Entidades
 - 5.1 Usuario
 - 5.2 Produto
 - 5.3 Categoria
 - 5.4 Movimentacao
 - 6. Camada DAO — Persistência
 - 6.1 UsuarioDAO
 - 6.2 ProdutoDAO
 - 6.3 CategoriaDAO
 - 6.4 MovimentacaoDAO
 - 7. Camada Controller — Regras de Negócio
 - 7.1 LoginController
 - 7.2 CadastroUsuarioController
 - 7.3 ProdutoController
 - 8. Camada View — Interface Gráfica
 - 8.1 MainView
 - 8.2 LoginView
 - 8.3 CadastroUsuarioView
 - 8.4 TelaPrincipalView
 - 9. Camada Util — Utilitários
 - 9.1 ConnectionUtil
 - 9.2 HashUtil
 - 9.3 SessaoUsuario
 - 10. Regras de Negócio
 - RN-01 — Autenticação Obrigatória
 - RN-02 — Senha Segura
 - RN-03 — Login Único
 - RN-04 — Transação em Movimentações
 - RN-05 — Estoque Atualizado Automaticamente
 - RN-06 — Categoria Obrigatória para Produtos
 - RN-07 — Confirmação de Exclusão

- [RN-08 — Histórico com LEFT JOIN](#)
- [RN-09 — Auto-Init do Banco](#)
- [RN-10 — Usuário Admin Padrão](#)
- [11. Script SQL \(init.sql\)](#)
 - [Criação do Banco](#)
 - [Tabelas](#)
 - [Dados Iniciais \(Seed\)](#)
- [12. Configuração do Projeto](#)
 - [pom.xml \(Maven\)](#)
 - [db.properties](#)
- [13. Fluxo de Navegação da Aplicação](#)
- [14. Glossário](#)

Documentação do Sistema de Controle de Estoque

Sumário

- [1. Visão Geral](#)
- [2. Topologia da Arquitetura](#)
- [3. Diagrama de Camadas](#)
- [4. Modelo de Dados \(MER\)](#)
- [5. Camada Model — Entidades](#)
- [6. Camada DAO — Persistência](#)
- [7. Camada Controller — Regras de Negócio](#)
- [8. Camada View — Interface Gráfica](#)
- [9. Camada Util — Utilitários](#)
- [10. Regras de Negócio](#)
- [11. Script SQL \(init.sql\)](#)
- [12. Configuração do Projeto \(pom.xml / db.properties\)](#)
- [13. Fluxo de Navegação da Aplicação](#)
- [14. Glossário](#)

1. Visão Geral

Sistema de Controle de Estoque é uma aplicação desktop Java com interface gráfica Swing e banco de dados MySQL. Permite:

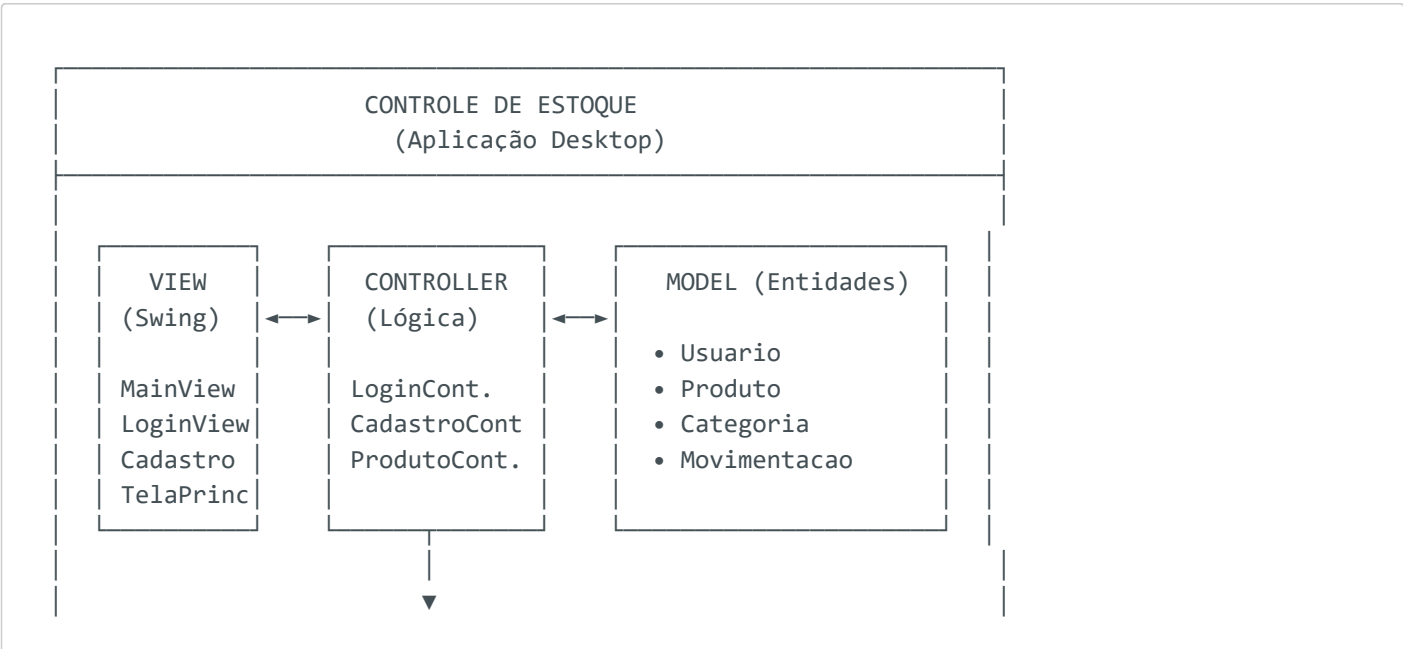
- Cadastro e autenticação de usuários
- CRUD de produtos (criar, ler, atualizar, excluir)
- Gerenciamento de categorias
- Registro de movimentações de entrada e saída
- Consulta de histórico de movimentações
- Controle de estoque em tempo real

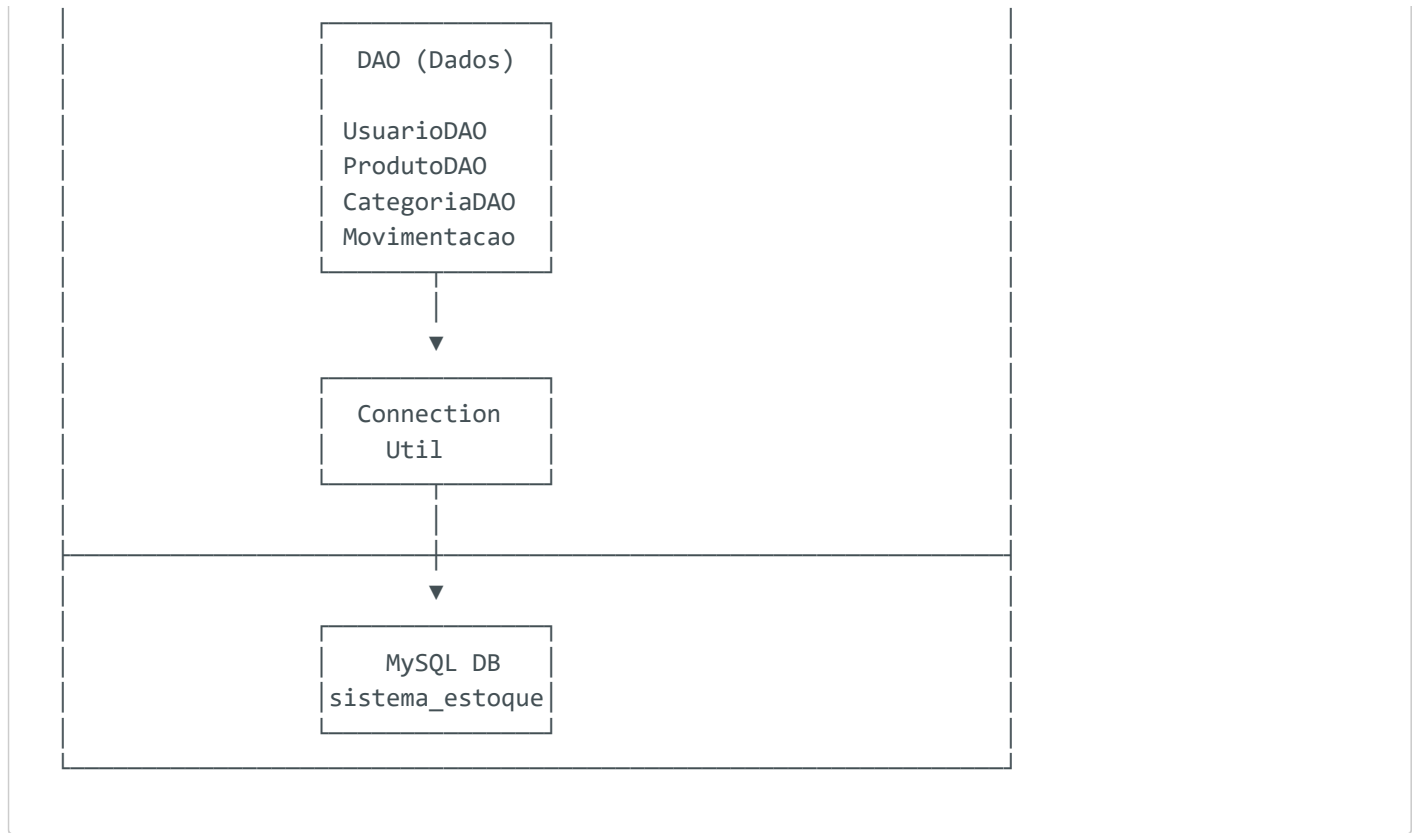
Tecnologias

Componente	Tecnologia
Linguagem	Java 11
Interface	Java Swing (NetBeans GUI Builder)
Banco	MySQL 8.0
Conector JDBC	mysql-connector-j 8.0.33
Build	Apache Maven
Gerenciamento	Maven (pom.xml)

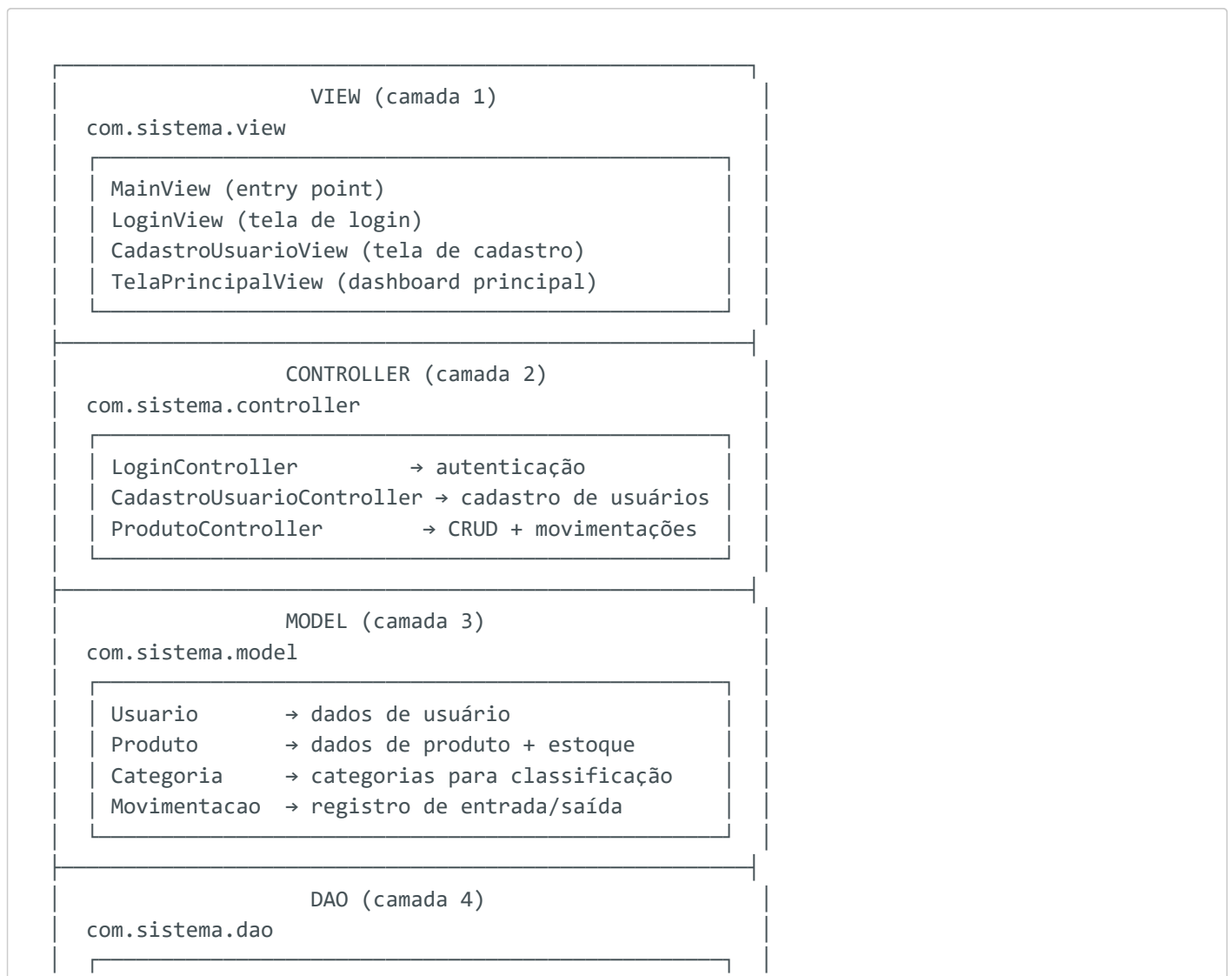
2. Topologia da Arquitetura

A aplicação segue o padrão **MVC (Model-View-Controller)** com uma camada adicional de persistência (DAO).





3. Diagrama de Camadas



UsuarioDAO	→ insert, buscar, autenticar
ProdutoDAO	→ CRUD completo
CategoriaDAO	→ listar, buscar, inserir
MovimentacaoDAO	→ registrar, listar histórico

UTIL (camada 5)

com.sistema.util

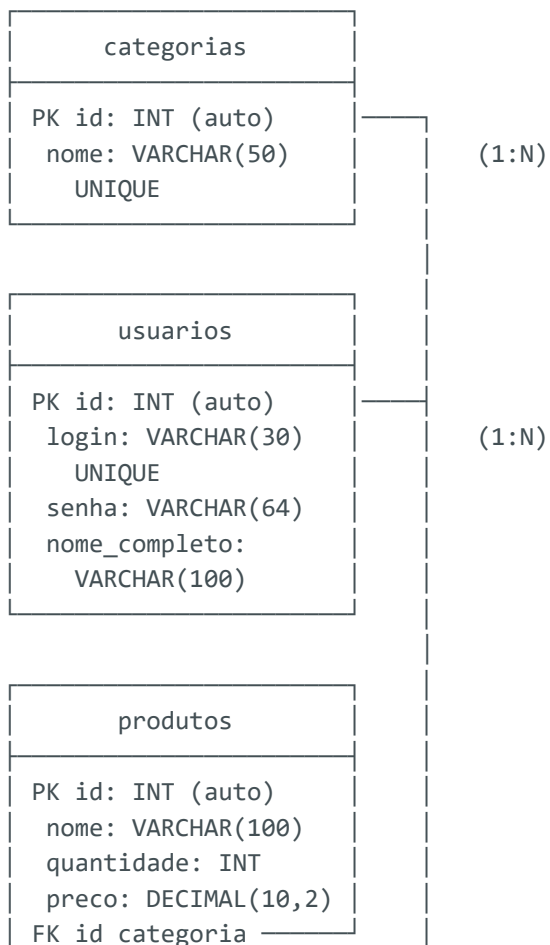
ConnectionUtil	→ conexão MySQL + auto-init DB
HashUtil	→ SHA-256 para senhas
SessaoUsuario	→ sessão do usuário logado

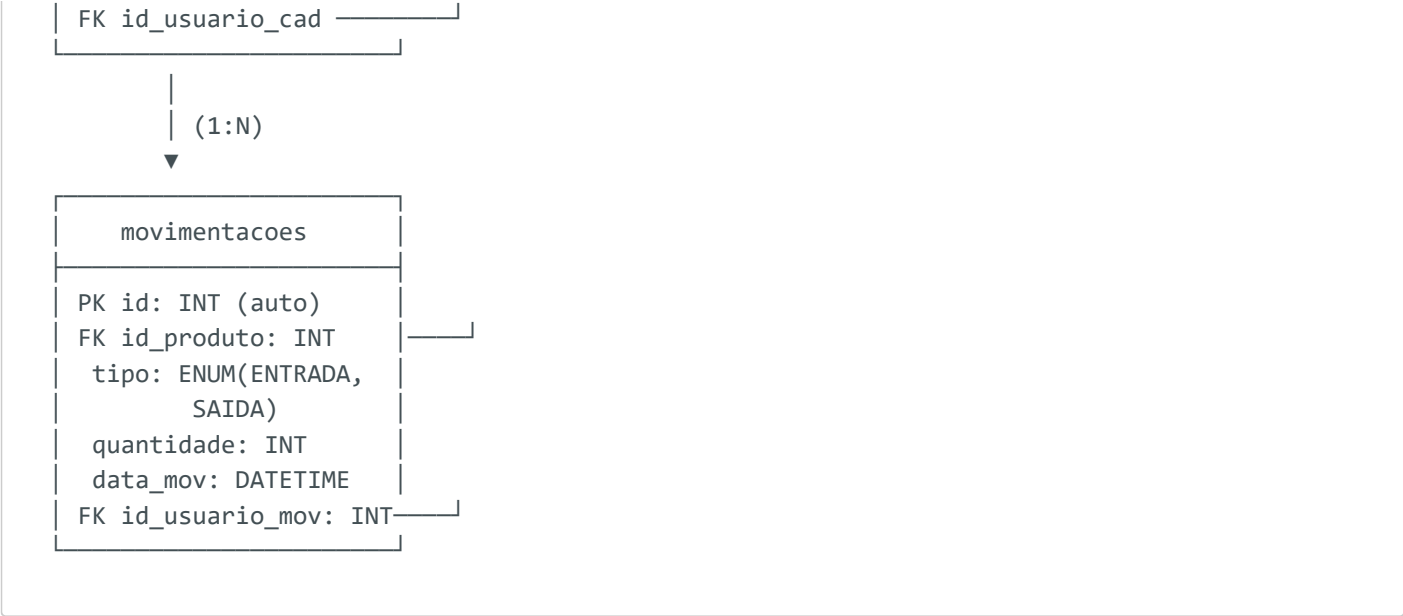
CONFIGURAÇÃO (resources)

pom.xml	→ dependências Maven
db.properties	→ URL, user, pass do MySQL
init.sql	→ script SQL para criar o banco

4. Modelo de Dados (MER)

Diagrama Entidade-Relacionamento





Dicionário de Dados

Tabela	Campo	Tipo	Restrições	Descrição
categorias	id	INT	PK, AUTO_INCREMENT	ID único da categoria
categorias	nome	VARCHAR(50)	NOT NULL, UNIQUE	Nome da categoria
usuarios	id	INT	PK, AUTO_INCREMENT	ID único do usuário
usuarios	login	VARCHAR(30)	NOT NULL, UNIQUE	Nome de login
usuarios	senha	VARCHAR(64)	NOT NULL	Hash SHA-256 da senha
usuarios	nome_completo	VARCHAR(100)		Nome completo para exibição
produtos	id	INT	PK, AUTO_INCREMENT	ID único do produto
produtos	nome	VARCHAR(100)	NOT NULL	Nome do produto
produtos	quantidade	INT	NOT NULL, DEFAULT 0	Quantidade em estoque
produtos	preco	DECIMAL(10,2)	NOT NULL	Preço unitário

Tabela	Campo	Tipo	Restrições	Descrição
produtos	id_categoria	INT	FK → categorias(id)	Categoria do produto
produtos	id_usuario_cad	INT	FK → usuarios(id)	Usuário que cadastrou
movimentacoes	id	INT	PK, AUTO_INCREMENT	ID único da movimentação
movimentacoes	id_produto	INT	NOT NULL, FK → produtos(id)	Produto movimentado
movimentacoes	tipo	ENUM	NOT NULL (ENTRADA, SAIDA)	Tipo de movimento
movimentacoes	quantidade	INT	NOT NULL	Quantidade movimentada
movimentacoes	data_mov	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data/hora do movimento
movimentacoes	id_usuario_mov	INT	FK → usuarios(id)	Usuário que registrou

5. Camada Model — Entidades

5.1 Usuario

Arquivo: `src/main/java/com/sistema/model/Usuario.java`

Representa um usuário do sistema.

Atributo	Tipo	Descrição
id	Integer	Identificador único (auto-incremento)
login	String	Nome de login (único no sistema)
senha	String	Hash SHA-256 da senha (64 caracteres)
nomeCompleto	String	Nome completo para exibição

Construtores:

- `Usuario()` — construtor padrão

- `Usuario(Integer id, String login, String senha, String nomeCompleto)` — construtor completo

Métodos:

- Getters e setters para todos os atributos
 - `toString()` → retorna `nomeCompleto` ou `login`
 - `equals()` / `hashCode()` → baseados no campo `id`
-

5.2 Produto

Arquivo: `src/main/java/com/sistema/model/Produto.java`

Representa um produto no estoque.

Atributo	Tipo	Descrição
id	Integer	Identificador único
nome	String	Nome do produto
quantidade	Integer	Quantidade atual em estoque
preco	BigDecimal	Preço unitário (precisão decimal)
categoria	Categoria	Objeto Categoria (relacionamento)
usuarioCad	Usuario	Usuário que cadastrou (relacionamento)

Construtores:

- `Produto()` — construtor padrão
- `Produto(Integer id, String nome, Integer quantidade, BigDecimal preco, Categoria categoria, Usuario usuarioCad)` — completo

Métodos:

- Getters e setters para todos os atributos
 - `toString()` → retorna o nome
 - `equals()` / `hashCode()` → baseados no campo `id`
-

5.3 Categoria

Arquivo: `src/main/java/com/sistema/model/Categoria.java`

Representa uma categoria para classificar produtos.

Atributo	Tipo	Descrição
id	Integer	Identificador único
nome	String	Nome da categoria (único)

Construtores:

- `Categoria()` — construtor padrão
- `Categoria(Integer id, String nome)` — completo

Métodos:

- Getters e setters
- `toString()` → retorna o nome
- `equals()` / `hashCode()` → baseados no campo `id`

5.4 Movimentacao

Arquivo: `src/main/java/com/sistema/model/Movimentacao.java`

Representa uma movimentação de entrada ou saída de estoque.

Atributo	Tipo	Descrição
id	Integer	Identificador único
produto	Produto	Produto movimentado
tipo	TipoMovimentacao (enum)	ENTRADA ou SAIDA
quantidade	Integer	Quantidade movimentada
dataMov	LocalDateTime	Data/hora do registro
usuarioMov	Usuario	Usuário que registrou

Enum:

- `TipoMovimentacao.ENTRADA` — adiciona ao estoque
- `TipoMovimentacao.SAIDA` — remove do estoque

Construtores:

- `Movimentacao()` — construtor padrão
- `Movimentacao(...)` — completo

Métodos:

- Getters e setters para todos os atributos
 - `equals()` / `hashCode()` → baseados no campo `id`
-

6. Camada DAO — Persistência

6.1 UsuarioDAO

Arquivo: `src/main/java/com/sistema/dao/UsuarioDAO.java`

Operações com a tabela `usuarios`.

Método	Descrição
<code>inserir(Usuario)</code>	Insere um novo usuário no banco.
<code>buscarPorLogin(String)</code>	Busca usuário pelo login. Retorna <code>Optional<Usuario></code> .
<code>autenticar(String, String)</code>	Valida login + senha hash. Retorna <code>Optional<Usuario></code> .

6.2 ProdutoDAO

Arquivo: `src/main/java/com/sistema/dao/ProdutoDAO.java`

Operações com a tabela `produtos`. As consultas usam JOIN com `categorias` e `usuarios`.

Método	Descrição
<code>inserir(Produto)</code>	Insere um novo produto.
<code>atualizar(Produto)</code>	Atualiza nome, quantidade, preço e categoria.
<code>excluir(int id)</code>	Exclui um produto pelo ID.
<code>listar()</code>	Retorna todos os produtos (ordenados por nome).
<code>buscarPorNome(String)</code>	Busca produtos por nome parcial (LIKE %%).

6.3 CategoriaDAO

Arquivo: `src/main/java/com/sistema/dao/CategoriaDAO.java`

Operações com a tabela `categorias`.

Método	Descrição
<code>listar()</code>	Retorna todas as categorias ordenadas.
<code>buscarPorId(int)</code>	Busca categoria por ID.
<code>inserir(Categoria)</code>	Insere nova categoria (nome único).

6.4 MovimentacaoDAO

Arquivo: `src/main/java/com/sistema/dao/MovimentacaoDAO.java`

Operações com a tabela `movimentacoes`.

Método	Descrição
<code>registrar(Movimentacao)</code>	Registra movimentação + atualiza estoque (transação).
<code>listarPorProduto(int)</code>	Histórico de movimentações de um produto específico.
<code>listar()</code>	Histórico completo de movimentações (LEFT JOIN).

7. Camada Controller — Regras de Negócio

7.1 LoginController

Arquivo: `src/main/java/com/sistema/controller/LoginController.java`

Controla a tela de login.

Método	Descrição
<code>autenticar()</code>	Valida campos, gera hash SHA-256, busca no DAO, inicia sessão
<code>abrirCadastro()</code>	Navega para a tela de cadastro de novo usuário
<code>abrirDashboard()</code>	Abre a tela principal após login bem-sucedido

Regras de negócio aplicadas:

- Login e senha são obrigatórios

- Senha é hasheada com SHA-256 antes de comparar com o banco
 - Usuário logado é armazenado em `SessaoUsuario` para uso nas demais operações
-

7.2 CadastroUsuarioController

Arquivo: `src/main/java/com/sistema/controller/CadastroUsuarioController.java`

Controla a tela de cadastro de novos usuários.

Método	Descrição
<code>cadastrar()</code>	Valida campos, verifica duplicidade, hasheia senha, insere
<code>voltarLogin()</code>	Retorna à tela de login

Regras de negócio aplicadas:

- Login e senha são obrigatórios
 - Login deve ser único no sistema (verificação antes de inserir)
 - Senha é armazenada como hash SHA-256
 - O campo `nomeCompleto` recebe o mesmo valor do `login` no cadastro simplificado
-

7.3 ProdutoController

Arquivo: `src/main/java/com/sistema/controller/ProdutoController.java`

Controlador principal do dashboard. Gerencia todas as operações de produto, categoria e movimentação.

Método	Descrição
<code>buscarProdutos()</code>	Dispara busca textual na tabela de produtos
<code>salvarProduto(boolean novo)</code>	Cadastra ou atualiza produto
<code>excluirProduto()</code>	Exclui produto selecionado (com confirmação)
<code>registrarMovimentacao(Tipo)</code>	Registra entrada ou saída de estoque
<code>cadastrarCategoria()</code>	Cadastra nova categoria

Regras de negócio aplicadas no CRUD:

- Nome, quantidade e preço são obrigatórios

- Preço aceita vírgula como separador decimal (convertido para ponto)
 - Categoria deve ser selecionada no combobox
 - Para atualizar, um produto deve estar selecionado na tabela
 - Exclusão requer confirmação do usuário
-

8. Camada View — Interface Gráfica

8.1 MainView

Arquivo: `src/main/java/com/sistema/view/MainView.java`

Classe de entrada da aplicação. Inicializa a `LoginView` no Event Dispatch Thread do Swing e a conecta ao `LoginController`.

Método principal:

- `main(String[] args)` → ponto de entrada da aplicação
-

8.2 LoginView

Arquivo: `src/main/java/com/sistema/view/LoginView.java`

Tela de login com campos de usuário e senha e botões "Entrar" e "Cadastrar-se".

Métodos ponte (para acesso do Controller):

Método	Retorno	Descrição
<code>getLogin()</code>	String	Texto do campo de login
<code>getSenha()</code>	String	Senha digitada
<code>getBtnEntrar()</code>	JButton	Botão "Entrar"
<code>getBtnCriarConta()</code>	JButton	Botão "Cadastrar-se"
<code>limparCampos()</code>	void	Limpa campos de login/senha

8.3 CadastroUsuarioView

Arquivo: `src/main/java/com/sistema/view/CadastroUsuarioView.java`

Tela de cadastro com campos de novo login e senha, botões "Salvar Cadastro" e "Voltar".

Métodos ponte:

Método	Retorno	Descrição
<code>getNovoLogin()</code>	String	Texto do campo de novo login
<code>getNovaSenha()</code>	String	Nova senha digitada
<code>getBtnSalvarCadastro()</code>	JButton	Botão "Salvar Cadastro"
<code>getBtnVoltar()</code>	JButton	Botão "Voltar para Login"
<code>limparCampos()</code>	void	Limpa campos

8.4 TelaPrincipalView

Arquivo: `src/main/java/com/sistema/view/TelaPrincipalView.java`

Dashboard principal com três abas (JTabbedPane):

- 1. **Produtos** — busca, tabela de produtos, formulário CRUD
- 2. **Categorias** — tabela de categorias + cadastro rápido
- 3. **Movimentações** — histórico geral + botões de entrada/saída

A aba de movimentações possui um painel inferior com campo de quantidade e botões "Registrar Entrada" e "Registrar Saída" adicionados manualmente (código extra na linha 395-406).

Métodos ponte por aba:

Método	Retorno	Descrição
<code>getTxtBusca()</code>	String	Campo de busca por nome
<code>getBtnBuscar()</code>	JButton	Botão "Buscar"
<code>getTableProdutos()</code>	JTable	Tabela de produtos
<code>getTxtNome()</code>	String	Campo nome do produto
<code>getTxtQuantidade()</code>	String	Campo quantidade
<code>getTxtPreco()</code>	String	Campo preço
<code>getCbCategoria()</code>	JComboBox	Combo de categorias
<code>getBtnCadastrar()</code>	JButton	Botão "Cadastrar"

Método	Retorno	Descrição
<code>getBtnAtualizar()</code>	JButton	Botão "Atualizar"
<code>getBtnExcluir()</code>	JButton	Botão "Excluir"
<code>limparCamposProduto()</code>	void	Limpa formulário de produto
<code>getTableCategorias()</code>	JTable	Tabela de categorias
<code>getTxtNomeCategoria()</code>	String	Campo nome da categoria
<code>getBtnCadastrarCategoria()</code>	JButton	Botão cadastrar categoria
<code>getTableMovimentacoes()</code>	JTable	Tabela de histórico
<code>getBtnEntrada()</code>	JButton	Botão "Registrar Entrada"
<code>getBtnSaida()</code>	JButton	Botão "Registrar Saída"
<code>getTxtQtdMov()</code>	String	Campo quantidade movimentação
<code>limparCampoMovimentacao()</code>	void	Limpa campo de movimentação

9. Camada Util — Utilitários

9.1 ConnectionUtil

Arquivo: `src/main/java/com/sistema/util/ConnectionUtil.java`

Gerencia a conexão com o banco de dados MySQL.

Método	Descrição
<code>getConnection()</code>	Retorna conexão singleton. Cria se não existir ou estiver fechada.
<code>initDatabase(Connection)</code>	Cria tabelas e insere dados iniciais (admin + categorias).
<code>close()</code>	Fecha a conexão com o banco.

Funcionamento:

- Lê configurações do arquivo `db.properties` no classpath
- Carrega o driver JDBC `com.mysql.cj.jdbc.Driver`
- Conecta ao MySQL com URL, usuário e senha do properties
- Na primeira conexão, executa `initDatabase` que cria as 4 tabelas (se não existirem) e insere dados seed

9.2 HashUtil

Arquivo: `src/main/java/com/sistema/util/HashUtil.java`

Utilitário para hash de senhas com SHA-256.

Método	Descrição
<code>sha256(String)</code>	Retorna o hash SHA-256 em hexadecimal (64 chars)

9.3 SessaoUsuario

Arquivo: `src/main/java/com/sistema/util/SessaoUsuario.java`

Mantém o usuário autenticado durante a sessão (singleton estático).

Método	Descrição
<code>`set(Usuario)</code>	Define o usuário logado
<code>get()</code>	Retorna o usuário logado
<code>clear()</code>	Remove o usuário da sessão (logout)
<code>isLogado()</code>	Verifica se há um usuário logado

10. Regras de Negócio

RN-01 — Autenticação Obrigatória

Para acessar o sistema, o usuário deve fornecer login e senha válidos. A senha é verificada através de hash SHA-256.

RN-02 — Senha Segura

Nenhuma senha é armazenada em texto puro. O sistema gera o hash SHA-256 antes de qualquer operação de escrita ou leitura no banco.

RN-03 — Login Único

Cada login deve ser único no sistema. O cadastro de um novo usuário verifica duplicidade antes de inserir.

RN-04 — Transação em Movimentações

O registro de uma movimentação e a atualização do estoque são feitos em uma única transação atômica. Se qualquer parte falhar, um rollback é executado, garantindo a consistência dos dados.

RN-05 — Estoque Atualizado Automaticamente

- **Entrada:** a quantidade do produto é incrementada ($\text{quantidade} + \text{qtd}$)
- **Saída:** a quantidade do produto é decrementada ($\text{quantidade} - \text{qtd}$)

RN-06 — Categoria Obrigatória para Produtos

Todo produto deve pertencer a uma categoria. O sistema exige seleção no combobox antes de cadastrar ou atualizar.

RN-07 — Confirmação de Exclusão

A exclusão de um produto requer confirmação explícita do usuário via diálogo de confirmação.

RN-08 — Histórico com LEFT JOIN

As consultas de movimentações usam LEFT JOIN para garantir que o histórico permaneça visível mesmo se o produto ou usuário associado forem removidos futuramente.

RN-09 — Auto-Init do Banco

Na primeira conexão, o sistema cria automaticamente as tabelas necessárias e insere dados iniciais (admin:admin e 3 categorias padrão).

RN-10 — Usuário Admin Padrão

O sistema já vem com um usuário administrador pré-cadastrado:

- Login: `admin`
 - Senha: `admin`
 - O hash armazenado é:
`8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918`
-

11. Script SQL (init.sql)

Arquivo: `src/main/resources/init.sql` (independente) e lógica embutida em `ConnectionUtil.java`

Criação do Banco

```
CREATE DATABASE IF NOT EXISTS sistema_estoque
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE sistema_estoque;
```

Tabelas

```
CREATE TABLE IF NOT EXISTS categorias (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE IF NOT EXISTS usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  login VARCHAR(30) NOT NULL UNIQUE,
  senha VARCHAR(64) NOT NULL,
  nome_completo VARCHAR(100)
);

CREATE TABLE IF NOT EXISTS produtos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  quantidade INT NOT NULL DEFAULT 0,
  preco DECIMAL(10,2) NOT NULL,
  id_categoria INT,
  id_usuario_cad INT,
  FOREIGN KEY (id_categoria) REFERENCES categorias(id),
  FOREIGN KEY (id_usuario_cad) REFERENCES usuarios(id)
);

CREATE TABLE IF NOT EXISTS movimentacoes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  id_produto INT NOT NULL,
  tipo ENUM('ENTRADA','SAIDA') NOT NULL,
```

```
quantidade INT NOT NULL,  
data_mov DATETIME DEFAULT CURRENT_TIMESTAMP,  
id_usuario_mov INT,  
FOREIGN KEY (id_produto) REFERENCES produtos(id),  
FOREIGN KEY (id_usuario_mov) REFERENCES usuarios(id)  
);
```

Dados Iniciais (Seed)

```
INSERT IGNORE INTO usuarios (login, senha, nome_completo)  
VALUES ('admin', '8c6976e5b5410415bde908bd4dee15dfb167a9c873fc4bb8a81f6f2ab448a918',  
'Administrador');  
-- senha: admin  
  
INSERT IGNORE INTO categorias (nome)  
VALUES ('Eletrônicos'), ('Escritório'), ('Limpeza');
```

12. Configuração do Projeto

pom.xml (Maven)

Localização: `pom.xml`

```
<groupId>com.sistema</groupId>  
<artifactId>controle-estoque</artifactId>  
<version>1.0.0</version>  
<packaging>jar</packaging>
```

Propriedades:

- Java 11 (source e target)
- UTF-8

Dependências:

- `mysql-connector-j` 8.0.33 — driver JDBC MySQL

Plugins:

- `maven-compiler-plugin` 3.11.0
- `exec-maven-plugin` 3.1.0 (mainClass: `com.sistema.view.MainView`)

Comandos Maven:

Comando	Descrição
<code>mvn clean compile</code>	Compila o projeto
<code>mvn exec:java</code>	Executa a aplicação
<code>mvn package</code>	Gera o JAR executável

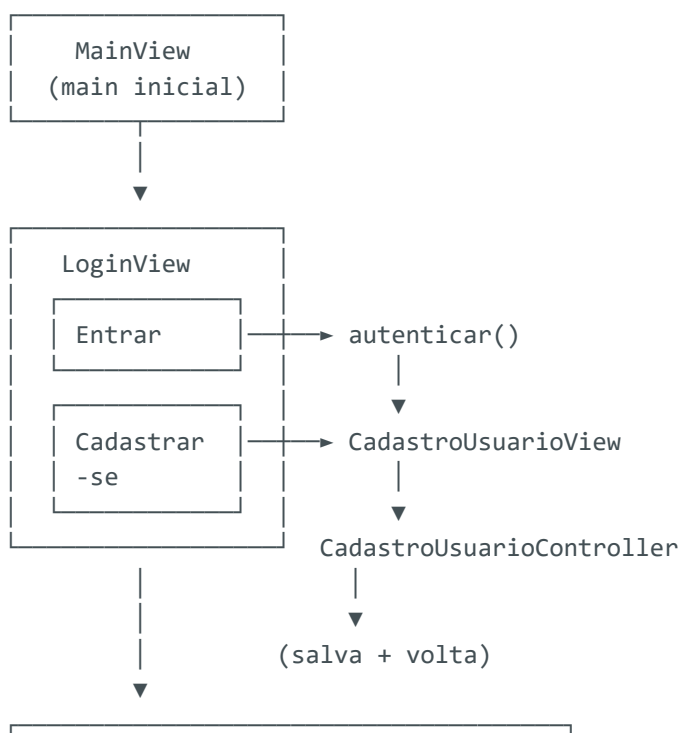
db.properties

Localização: `src/main/resources/db.properties`

```
db.url=jdbc:mysql://localhost:3306/sistema_estoque?  
useSSL=false&serverTimezone=UTC&createDatabaseIfNotExist=true  
db.user=root  
db.pass=
```

- **db.url:** URL JDBC com parâmetros para desabilitar SSL, definir timezone e criar banco automaticamente
- **db.user:** Usuário do MySQL (root por padrão)
- **db.pass:** Senha do MySQL (vazia por padrão — ajustar conforme ambiente)

13. Fluxo de Navegação da Aplicação





14. Glossário

Termo	Definição
CRUD	Create, Read, Update, Delete — operações básicas.
DAO	Data Access Object — padrão para persistência em BD.
DTO	Data Transfer Object — objeto para transporte de dados.
EDT	Event Dispatch Thread — thread única do Swing para UI.
ENUM	Tipo enumerado com valores fixos (ENTRADA, SAIDA).
Hash	Função unidirecional que transforma texto em código.
JDBC	Java Database Connectivity — API de conexão com BD.
JFrame	Janela principal do Swing.
JOptionPane	Diálogo de mensagens do Swing (alertas, confirmação).
JTable	Componente Swing para exibição de tabelas.
LEFT JOIN	JOIN que mantém registros mesmo sem correspondência.
Maven	Ferramenta de build e gerenciamento de dependências.
MVC	Model-View-Controller — padrão de arquitetura.
SHA-256	Algoritmo de hash seguro de 256 bits.
Singleton	Padrão que garante uma única instância de um objeto.

Termo	Definição
Swing	Toolkit de interface gráfica do Java.